

ReloGo: Enterprise Engineering

Directives & Agent Handoffs

To the AI Developer: You are receiving a subset of this master specification. Execute ONLY the directives assigned to your Agent ID. Do not make assumptions outside of these parameters. Strict adherence to data privacy (PIPEDA) and client-side processing is mandatory.

AGENT 1: BACKEND & DATABASE ARCHITECT (SUPABASE)

Context: You are responsible for the foundational data layer. We are using Supabase (PostgreSQL).

Execution: Generate the raw SQL migration scripts to build this exact schema.

1. Database Schema Requirements (Strict Data Types)

Table 1: global_tasks

- id: UUID (Primary Key, default gen_random_uuid())
- task_key: VARCHAR(100) (Unique, e.g., 'EXCHANGE_DRIVERS_LICENCE')
- title_en: VARCHAR(255)
- base_description_en: TEXT
- created_at: TIMESTAMPTZ (default NOW())

Table 2: corridor_task_rules (The provincial mapping)

- id: UUID (Primary Key)
- task_id: UUID (Foreign Key to global_tasks(id))
- origin_province: VARCHAR(2) (e.g., 'ON', 'ANY')
- dest_province: VARCHAR(2) (e.g., 'AB')
- days_deadline: INTEGER (Number of days after move to complete)
- is_mandatory: BOOLEAN (Default TRUE)

Table 3: official_sources

- id: UUID (Primary Key)
- corridor_rule_id: UUID (Foreign Key to corridor_task_rules(id))
- agency_name: VARCHAR(255)
- official_url: TEXT
- last_verified: TIMESTAMPTZ

Table 4: rule_change_alerts (For the Scraper)

- id: UUID (Primary Key)

- official_source_id: UUID (Foreign Key to official_sources(id))
- old_hash: VARCHAR(255)
- new_hash: VARCHAR(255)
- diff_summary: TEXT
- status: VARCHAR(50) (CHECK: 'PENDING', 'APPROVED', 'DISMISSED')
- created_at: TIMESTAMPTZ

Table 5: user_profiles (Managed by Supabase Auth)

- id: UUID (Foreign Key to auth.users(id), Primary Key)
- move_date: DATE
- origin_prov: VARCHAR(2)
- dest_prov: VARCHAR(2)
- has_vehicle: BOOLEAN
- has_dependents: BOOLEAN

Table 6: user_task_progress

- user_id: UUID (Foreign Key to user_profiles(id))
- task_rule_id: UUID (Foreign Key to corridor_task_rules(id))
- status: VARCHAR(20) (CHECK: 'LOCKED', 'AVAILABLE', 'COMPLETED')
- Primary Key is composite: (user_id, task_rule_id)

2. Row Level Security (RLS) Policies

- **Public Tables:** global_tasks, corridor_task_rules, official_sources must have SELECT policies for anon and authenticated roles. NO INSERT/UPDATE access.
- **Private Tables:** user_profiles and user_task_progress must have policies where auth.uid() = id (or user_id). Users can only read, insert, and update their own rows.
- **Admin Tables:** rule_change_alerts is strictly service_role only.

Agent 1 Definition of Done:

Output a single, massive 001_init.sql file containing all table creations, foreign keys, RLS enablement, RLS policies, and 5 rows of seed data for the ON->AB corridor.

AGENT 2: MOBILE FRONTEND ARCHITECT (EXPO / REACT NATIVE)

Context: You are building the iOS/Android client.

Stack: Expo SDK 50+, Expo Router, TypeScript, NativeWind (Tailwind), supabase-js, expo-secure-store, pdf-lib.

1. Exact Folder Structure (Expo Router)

/app

/_layout.tsx (Initializes Supabase session & Auth provider)
/index.tsx (Redirects to /onboarding or /(tabs))
/(auth)
 /onboarding.tsx (Form: origin, dest, date, vehicle, kids)
/(tabs)
 /_layout.tsx (Bottom Tab Navigator)
 /checklist.tsx (The main DAG task list)
 /profile.tsx (Includes "Delete My Data" button)
/task
 /[id].tsx (Dynamic task detail screen)
/lib
 /supabase.ts (Client init with async storage)
 /pdfEngine.ts (The pdf-lib execution script)

2. Strict Technical Mandates

- **PII Storage:** Health card numbers, driver's licences, and exact street addresses **MUST** NEVER touch the Supabase client. You must write a wrapper around expo-secure-store to save this locally on the device.
- **The Checklist Logic:** On app/(tabs)/checklist.tsx, fetch the user's origin/dest from user_profiles. Then fetch all matching corridor_task_rules. Render them in a FlatList. Filter out vehicle tasks if has_vehicle is false.
- **The PDF Engine (/lib/pdfEngine.ts):** Write an async function that accepts a task ID, fetches the local PII from secure-store, loads a local static PDF asset, uses pdf-lib to map the text into the form fields, saves it to FileSystem.cacheDirectory, and calls Sharing.shareAsync().
- **Data Deletion:** The profile screen **MUST** have a red button that calls SecureStore.deleteItemAsync() for all keys, and triggers a Supabase Edge Function to delete the auth.users record.

Agent 2 Definition of Done:

Output package.json with all dependencies, the lib/supabase.ts setup, and the complete code for checklist.tsx and pdfEngine.ts.

AGENT 3: SCRAPER & ADMIN ARCHITECT (RAILWAY)

Context: You are building the automated backend worker and the admin dashboard.

Stack: Python 3.11, Playwright (Scraper) AND React/Vite (Admin Dashboard).

Part A: Python Playwright Scraper (worker/main.py)

1. **Init:** Connect to Supabase using the SUPABASE_SERVICE_ROLE_KEY (bypassing RLS).
2. **Fetch:** Select all rows from official_sources.
3. **Scrape:** Loop through each URL using Playwright (Headless). Extract the document.body.innerText.
4. **Hash Logic:** Create a SHA-256 hash of the extracted text.
5. **Compare:** If the new hash matches the last_known_hash in the database, do nothing.
6. **Alert:** If the hash is different, DO NOT update the live rules. Execute an INSERT into rule_change_alerts with the old_hash, new_hash, and status 'PENDING'.

Part B: Admin Dashboard (admin/)

1. **Setup:** Initialize a React SPA (Vite + TypeScript + Tailwind).
2. **Auth:** This must be locked behind Supabase Email/Password auth. Only accounts with a specific admin email can log in.
3. **UI:** Build a Data Table reading from rule_change_alerts where status is 'PENDING'.
4. **Actions:** Provide two buttons per row: "Mark Reviewed (Dismiss)" (updates status to DISMISSED) and "Edit Global Rules" (opens a modal to manually update the corridor_task_rules table).

Agent 3 Definition of Done:

Output worker/main.py, worker/requirements.txt, and the primary AdminDashboard.tsx React component.



AGENT 4: WEB LANDING PAGE ARCHITECT (VERCEL)

Context: You are building the static marketing and waitlist page to capture leads and direct users to the App Stores.

Stack: Next.js 14 (App Router), TypeScript, Tailwind CSS, lucide-react (icons).

Requirement: Must be fully compatible with output: 'export' in next.config.js.

1. Component Architecture

- **Navbar:** Logo (ReloGo), "How it Works", "Waitlist" CTA.
- **Hero Section:**
 - *Headline: "The Canadian Relocation Autopilot."*
 - *Subheadline: "Moving between provinces? We auto-fill your health card, vehicle registration, and licence paperwork directly on your phone. Zero data leaves your device."*
- **The Hook (Interactive Form):**

- A dynamic component with two <select> dropdowns: "Moving From" (List all 13 provinces/territories) and "Moving To".
- When both are selected, reveal an email <input> and a "Join Waitlist" button.
- **Data Push:** Wire the "Join Waitlist" button to insert the email, origin, and destination into a Supabase waitlist table.

2. Design System Constraints

- Use a clean, high-trust color palette (e.g., Slate-900 for text, Blue-600 for primary buttons, red/white subtle accents for Canadian identity).
- Ensure the page is fully responsive (mobile-first design).

Agent 4 Definition of Done:

Output app/page.tsx (containing the Hero and Waitlist logic), tailwind.config.ts, and the next.config.js set for static export.